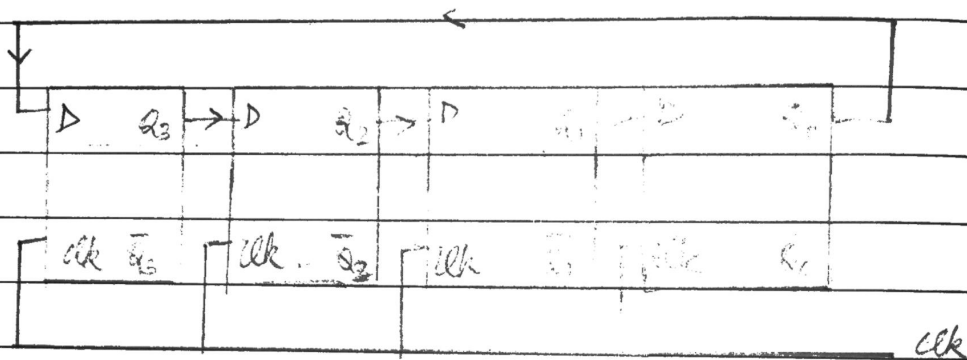


Lab No. 9

Counters

1. Write behavioural verilog code to implement a 4-bit ring counter



Table

clk	reset	Q_3	Q_2	Q_1	Q_0
x	1	1	0	0	0
1	0	0	1	0	0
2	0	0	0	1	0
3	0	0	0	0	1
4	0	1	0	0	0

code

```

module lab9q1(G,R,T);
input G, R;
output [3:0]T; sig[3:0]T;
always @ (posedge G or posedge R)
begin
if (R)
    T <= 4'b 1000;
else
begin
    T[3] <= T[0];
    T[2] <= T[3];
    T[1] <= T[2];
    T[0] <= T[1];
end
end
endmodule

```

Testbench

```

~ timescale 1ns/1ns;
~ include "lab9q1.v"

```

```

module lab9q1_tb;
sig G, R;

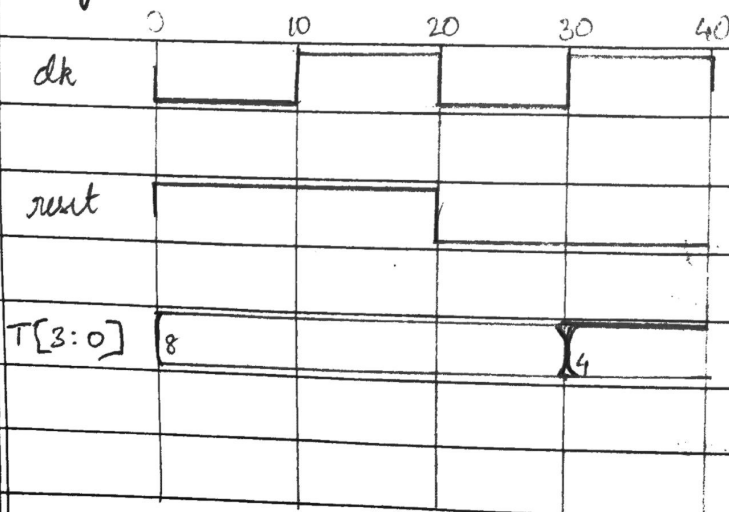
```

```

wire [3:0] T;
lab9q1 dup(C, R, T);
input C, R; initial begin
    { $dumpfile ("lab9q1.tb.vcd"); }
    $dumpvars (0, lab9q1.tb);
    C=0;
    forever #10 C=~C;
end
initial begin
    R=1; #20;
    R=0; #30;
    $display("Test Complete");
    $finish;
end
endmodule

```

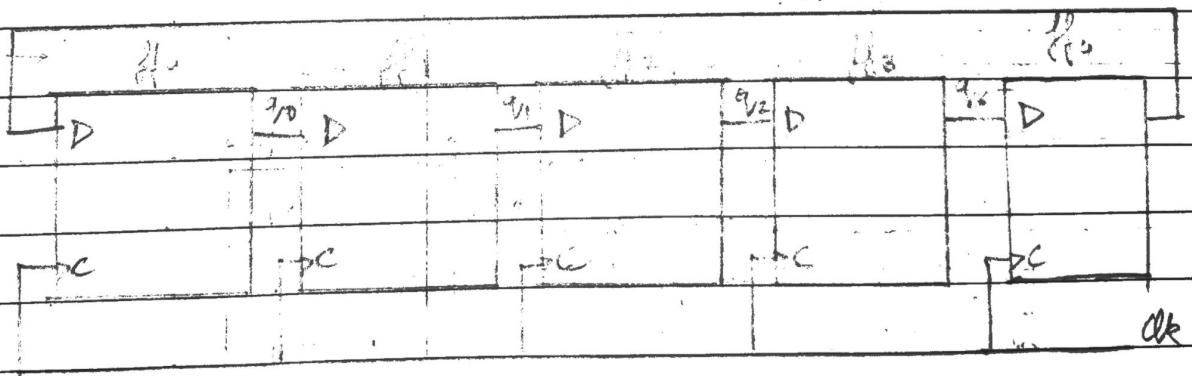
Waveform



Q. Design & ~~simulate~~ simulate 5-bit Johnsons counter.

T_4	T_3	T_2	T_1	T_0	AND gate	
0	0	0	0	0	$T_4' T_0'$	$\rightarrow Q_0$
1	0	0	0	0	$T_4' T_3'$	$\rightarrow Q_1$
1	1	0	0	0	$T_3 T_2'$	$\rightarrow Q_2$
1	1	1	0	0	$T_2 T_1'$	$\rightarrow Q_3$
1	1	1	1	0	$T_1 T_0'$	$\rightarrow Q_4$
1	1	1	1	1	$T_4 T_0$	$\rightarrow Q_5$
0	1	1	1	1	$T_4' T_3$	$\rightarrow Q_6$
0	0	1	1	1	$T_3' T_2$	$\rightarrow Q_7$
0	0	0	1	1	$T_2' T_1$	$\rightarrow Q_8$
0	0	0	0	1	$T_1' T_0$	$\rightarrow Q_9$

Diagram



code

```
module lab9a2 DFF (D, C, R, Q);  
    input D, C, R;  
    output reg Q;  
    always @ (posedge C or posedge R)  
    begin  
        if (R)  
            Q <= 0;  
        else  
            Q <= D;  
        end  
    endmodule
```

```
module lab9a2(C, R, T);  
    input C, R;  
    output [4:0] T;  
    DFF d4 (~T[0], C, R, T[4]);  
    DFF d3 ( T[4], C, R, T[3]);  
    DFF d2 (~T[3], C, R, T[2]);  
    DFF d1 ( T[2], C, R, T[1]);  
    DFF d0 ( T[1], C, R, T[0]);  
endmodule
```

Testbench

~ timescale 1ns/1ns

~ include "lab9q2.v"

```
module lab9q2_tb;
```

```
reg C, R;
```

```
wire [4:0] T;
```

```
lab9q2 dup(C, R, T);
```

```
initial begin
```

```
    { $dumpfile("lab9q2_tb.vcd");  
      $dumpvars(0, lab9q2_tb); }
```

```
    C = 0;
```

```
    forever #10 C = ~C;
```

```
    end
```

```
initial begin
```

```
    R = 1; #20;
```

```
    R = 0; #20;
```

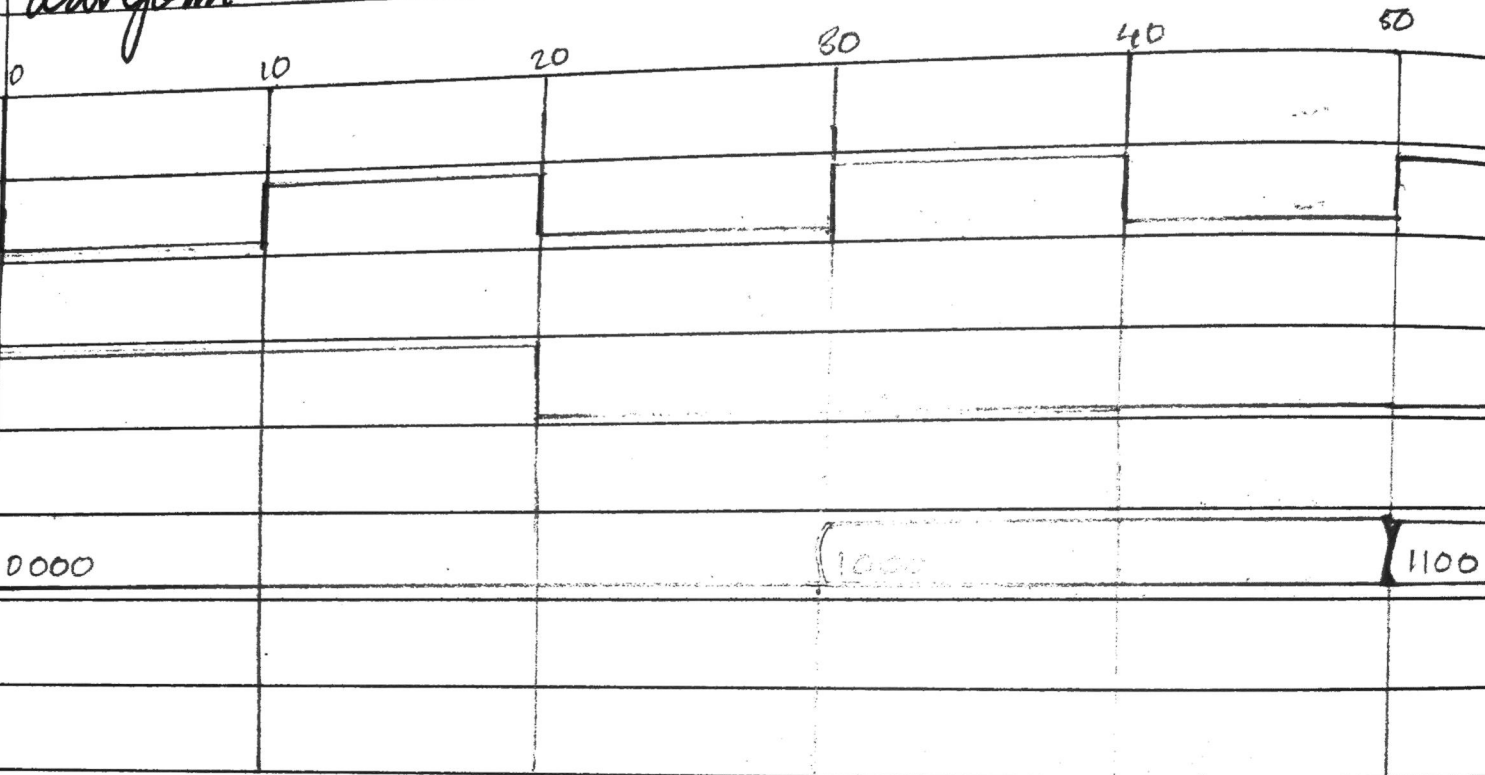
```
    $display("Test complete");
```

```
    $finish;
```

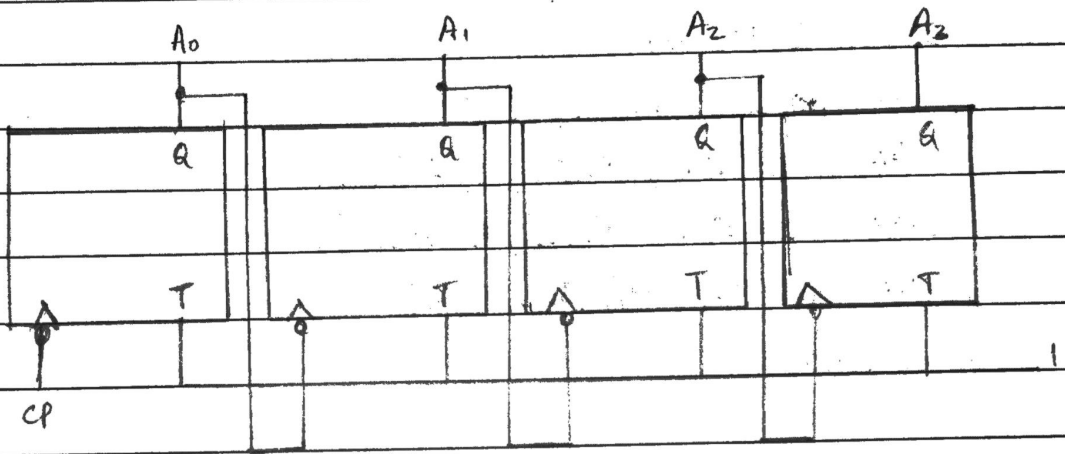
```
end
```

```
endmodule
```

Waveform



3. Write a Verilog to implement 4-bit asynchronous up-counter.



Code

```

module TFF (T, C, R, Q);
    input T, C, R;
    output reg Q;
    always @ (posedge C or posedge R)
    begin
        if (R)
            Q <= 0;
        else
            Q = T ? ~Q : Q;
    end
endmodule

```

```

module lab9q3 lab9q3(c, R, A);
input c, R;
output [3:0] A;
TFF a0(1'b1, c, R, A[0]);
TFF a1(1'b1, ~A[0], R, A[1]);
TFF a2(1'b1, ~A[1], R, A[2]);
TFF a3(1'b1, ~A[2], R, A[3]);
endmodule

```

Testbench

```

~timescale 1ns/1ns
~include "lab9q3.v"

```

```

module lab9q3_tb;
reg c, R;
wire [3:0] A;
lab9q3 dup(c, R, A);
initial begin
    $dumpfile("lab9q3_tb.vcd");
    $dumpvars(0, lab9q3_tb);
    c = 0;
    forever #10 c = ~c;
end
initial begin
    R = 1; #20;

```

```
R=0; #20;  
$display("Test complete");  
$finish;
```

end

endmodule

Waveform

